

Proyecto Final SQL

Base de Datos de Steam

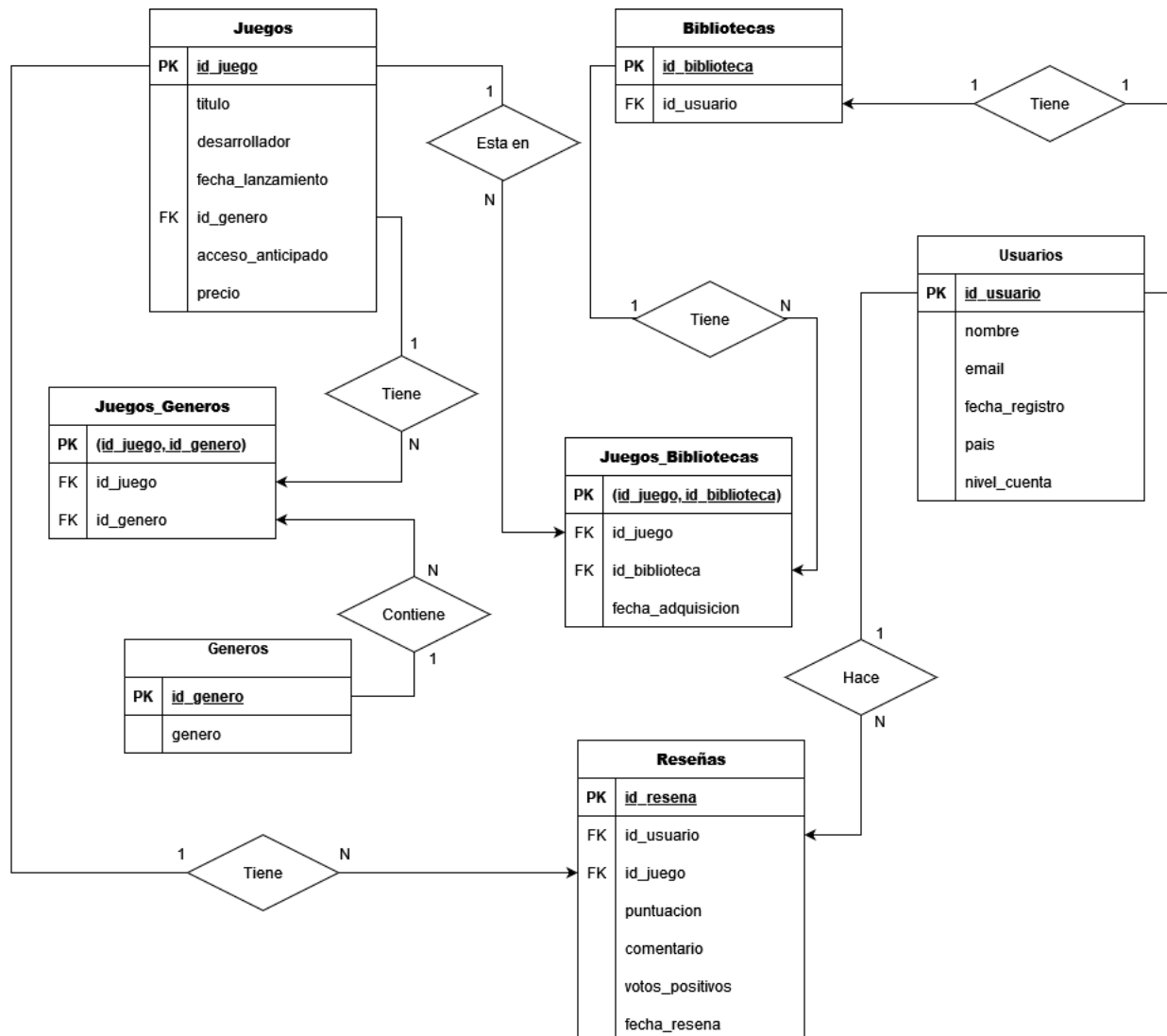
Melo Juan Manuel

04/09/2024

Descripción de la Temática de la Base de Datos

La base de datos que se desarrollará estará centrada en la plataforma de distribución de videojuegos Steam. Steam es una de las mayores plataformas de distribución digital de videojuegos en el mundo, ofreciendo una amplia variedad de títulos, desde juegos AAA hasta indie, y proporcionando a los usuarios una serie de servicios adicionales, como actualizaciones automáticas de juegos, almacenamiento en la nube, y una comunidad activa de jugadores.

Diagrama de entidad relación de la base de datos



Listado de Tablas

1. Juegos

Descripción: Almacena información sobre los videojuegos disponibles en Steam.

Campos:

id_juego (INT, PRIMARY KEY): Identificador único del juego.

titulo (VARCHAR, NOT NULL): Título del juego.

desarrollador (VARCHAR, NOT NULL): Nombre del desarrollador del juego.

fecha_lanzamiento (DATE, NOT NULL): Fecha de lanzamiento del juego.

id_genero (INT, FOREIGN KEY): Identificador del género.

acceso_anticipado (BOOLEAN, NOT NULL): Determina si el juego está en acceso anticipado.

precio (DECIMAL(7, 2), NOT NULL): Precio del juego.

2. Generos

Descripción: Almacena los diferentes géneros de videojuegos.

Campos:

id_genero (INT, PRIMARY KEY): Identificador único del género.

genero (VARCHAR, NOT NULL): Nombre del género.

3. Usuarios

Descripción: Contiene información sobre los usuarios de Steam.

Campos:

id_usuario (INT, PRIMARY KEY): Identificador único del usuario.

nombre (VARCHAR, NOT NULL): Nombre de usuario.

email (VARCHAR, UNIQUE, NOT NULL): Correo electrónico del usuario.

fecha_registro (DATE, NOT NULL): Fecha de registro del usuario.

pais (VARCHAR, NOT NULL): País del usuario.

nivel_cuenta (INT, NOT NULL): Nivel de la cuenta.

4. Bibliotecas

Descripción: Almacena las bibliotecas de juegos de los usuarios.

Campos:

id_biblioteca (INT, PRIMARY KEY): Identificador único de la biblioteca.

id_usuario (INT, FOREIGN KEY): Identificador del usuario propietario de la biblioteca.

5. Juegos Generos

Descripción: Relaciona los juegos con sus respectivos géneros.

Campos:

id_juego (INT, FOREIGN KEY): Identificador del juego.

id_genero (INT, FOREIGN KEY): Identificador del género.

PRIMARY KEY (id_juego, id_genero).

6. Juegos Bibliotecas

Descripción: Relaciona los juegos con las bibliotecas de los usuarios.

Campos:

id_juego (INT, FOREIGN KEY): Identificador del juego.

id_biblioteca (INT, FOREIGN KEY): Identificador de la biblioteca.

fecha_adquisicion (DATE, NOT NULL): Fecha de adquisición del juego.

PRIMARY KEY (id_juego, id_biblioteca).

7. Reseñas

Descripción: Almacena las reseñas de los juegos hechas por los usuarios.

Campos:

id_resena (INT, PRIMARY KEY): Identificador único de la reseña.

id_juego (INT, FOREIGN KEY): Identificador del juego reseñado.

id_usuario (INT, FOREIGN KEY): Identificador del usuario que hizo la reseña.

puntuacion (INT, NOT NULL): Calificación del juego.

comentario (TEXT): Comentario del usuario sobre el juego.

votos_positivos (INT, NOT NULL): Cantidad de votos positivos de la reseña.

fecha_resena (DATE, NOT NULL): Fecha en que se hizo la reseña.

Script en SQL de creación de la base de datos y tablas

En el siguiente link se comparte un repositorio github con el script en SQL de creación de la base de datos y tablas:

<https://github.com/juanmamelo/idea-melo.git>

Carga de Datos a las tablas

Se cargaron datos a las tablas creadas anteriormente mediante la función “Table Data Import Wizard” de MySQL Workbench usando CSVs generados con datos ficticios (creados en la web mockaroo.com)

Field Name	Type	Options
id_juego	Row Number	blank: 0 %
titulo	App Name	blank: 0 %
desarrollador	Fake Company Name	blank: 0 %
fecha_lanzamiento	Datetime	08/31/2023 to 08/31/2024 format: mm/dd/yyyy blank: 0 %
id_genero	Custom List	1,2,3,4,5,6,7,8,9 random blank: 0 %
acceso_anticipado	Boolean	blank: 0 %
precio	Money	between 0 and 10 in \$ blank: 0 %

+ ADD ANOTHER FIELD GENERATE FIELDS USING AI...

Rows: 1000 Format: CSV Line Ending: Unix (LF) Include: ☒ header ☐ BOM

Follow @mockarooDev

GENERATE DATA PREVIEW SAVE AS... DERIVE FROM EXAMPLE... MORE

Table Data Import

Configure Import Settings

Detected file format: csv 

Encoding: utf-8

Columns:

☒ Source Column	Dest Column
☒ id_juego	id_juego
☒ titulo	titulo
☒ desarrollador	desarrollador
☒ fecha_lanzamiento	fecha_lanzamient
☒ id_genero	id_genero
☒ acceso_anticipado	acceso_anticipad
☒ precio	precio

id_juego	titulo	desarrollador	fecha_lanz...	id_genero	acceso_an...	precio
1	Toughjoyfax	Hahn-Swift	2022-07-16	2	1	37.5
2	Voltsillam	Conroy-Har...	2017-02-21	1	1	58.99
3	Opela	D'Amore Inc	2015-05-01	2	1	17.35
4	Zamit	Hessel-Kuv...	2016-09-15	3	1	8.49
5	Alpha	Ondricka-Ni...	2019-09-18	4	1	39.84

< Back Next > Cancel

Se adjuntaron los CSVs al link:

<https://github.com/juanmamelos/idea-melos.git>

Listado de Vistas

1. vista juegos acceso anticipado: Esta vista muestra todos los juegos que están en acceso anticipado.

```
CREATE VIEW `steam_database`.`ranking_nivel_cuenta` AS
SELECT
  `steam_database`.`usuarios`.`nombre` AS `nombre`,
  `steam_database`.`usuarios`.`pais` AS `pais`,
```

```

        `steam_database`.`usuarios`.`nivel_cuenta` AS `nivel_cuenta`
FROM
        `steam_database`.`usuarios`
ORDER BY `steam_database`.`usuarios`.`nivel_cuenta` DESC

```

2. vista_juegos_generos: Esta vista muestra los juegos junto con sus géneros.

```

CREATE VIEW `steam_database`.`vista_juegos_generos` AS
SELECT
        `j`.`titulo` AS `Juego`,
        `g`.`genero` AS `Genero`,
        `j`.`desarrollador` AS `Desarrollador`,
        `j`.`fecha_lanzamiento` AS `Fecha de lanzamiento`,
        `j`.`precio` AS `Precio`
FROM
        (`steam_database`.`juegos` `j`
        JOIN `steam_database`.`generos` `g` ON ((`j`.`id_genero` = `g`.`id_genero`)))

```

3. vista_juegos_populares: Esta vista muestra los juegos junto con el número total de reseñas y la puntuación promedio.

```

CREATE VIEW `steam_database`.`vista_juegos_populares` AS
SELECT
        `j`.`titulo` AS `Juego`,
        `g`.`genero` AS `Genero`,
        `j`.`desarrollador` AS `Desarrollador`,
        COUNT(`r`.`id_resena`) AS TotalDeReseñas,
        AVG(r.puntuacion) AS PuntuacionPromedio
FROM
        ((steam_database.juegos j
        LEFT JOIN steam_database.reseñas r ON ((`j`.`id_juego` = `r`.`id_juego`)))

```

```

JOIN steam_database.generos g ON ((`j`.id_genero = `g`.id_genero)))
GROUP BY `j`.titulo , `g`.genero , `j`.desarrollador
ORDER BY PuntuacionPromedio DESC

```

4. vista_ranking_nivel_cuenta: Esta vista muestra el ranking de las cuentas por su nivel.

```

CREATE VIEW `steam_database`.`vista_ranking_nivel_cuenta` AS
SELECT
    `steam_database`.`usuarios`.`nombre` AS `Usuario`,
    `steam_database`.`usuarios`.`pais` AS `Pais`,
    `steam_database`.`usuarios`.`nivel_cuenta` AS `NivelDeCuenta`
FROM
    `steam_database`.`usuarios`
ORDER BY `steam_database`.`usuarios`.`nivel_cuenta` DESC

```

5. vista_resenas_detalladas: Esta vista muestra las reseñas con detalles del juego y del usuario que hizo la reseña.

```

CREATE VIEW `steam_database`.`vista_resenas_detalladas` AS
SELECT
    `j`.`titulo` AS `Juego`,
    `g`.`genero` AS `Genero`,
    `u`.`nombre` AS `Usuario`,
    `r`.`puntuacion` AS `Puntuacion`,
    `r`.`comentario` AS `Comentario`,
    `r`.`fecha_resena` AS FechaDeReseña,
    r.votos_positivos AS VotosPositivos
FROM
    (((steam_database.resenas r
    JOIN steam_database.juegos j ON ((`r`.id_juego = `j`.id_juego)))
    JOIN steam_database.generos g ON ((`j`.id_genero = `g`.id_genero)))

```



```
JOIN steam_database.usuarios u ON ((`r`.id_usuario = `u`.id_usuario)))
```

6. vista usuarios bibliotecas: Esta vista muestra los usuarios junto con los juegos que tienen en sus bibliotecas.

```
CREATE VIEW `steam_database`.`vista_usuarios_bibliotecas` AS  
SELECT  
    `u`.`nombre` AS `Username`,  
    `j`.`titulo` AS `Juego`,  
    `g`.`genero` AS `Genero`,  
    `jb`.`fecha_adquisicion` AS `Fecha de adquisicion`  
FROM  
    ((((`steam_database`.`usuarios` `u`  
    JOIN `steam_database`.`bibliotecas` `b` ON ((`u`.id_usuario = `b`.id_usuario)))  
    JOIN `steam_database`.`juegos_bibliotecas` `jb` ON ((`b`.id_biblioteca = `jb`.id_biblioteca)))  
    JOIN `steam_database`.`juegos` `j` ON ((`jb`.id_juego = `j`.id_juego)))  
    JOIN `steam_database`.`generos` `g` ON ((`j`.id_genero = `g`.id_genero)))
```

Funciones

ContarJuegosEnBiblioteca: Cuenta la cantidad de juegos en la biblioteca de un usuario. Interactua con las tablas Juegos_Bibliotecas y Bibliotecas.

```
CREATE FUNCTION `ContarJuegosEnBiblioteca`(id_usuario INT) RETURNS int  
DETERMINISTIC  
BEGIN  
    DECLARE conteo INT;  
    SELECT COUNT(jb.id_juego)  
    INTO conteo  
    FROM Juegos_Bibliotecas jb  
    JOIN Bibliotecas b ON jb.id_biblioteca = b.id_biblioteca  
    WHERE b.id_usuario = id_usuario;
```

```
    RETURN conteo;
END
```

ContarReseñasJuego: Esta función cuenta el número total de reseñas para un juego específico.

```
CREATE FUNCTION ContarReseñasJuego(fid_juego INT)
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE total_reseñas INT;
    SELECT COUNT(*)
    INTO total_reseñas
    FROM Reseñas
    WHERE fid_juego = id_juego;
    RETURN total_reseñas;
END;
```

Stored procedures

AgregarJuegoABiblioteca: Stored Procedure para Agregar un Juego a una Biblioteca. Interactua con la tabla Juegos_Bibliotecas.

```
CREATE PROCEDURE `AgregarJuegoABiblioteca`(
    IN p_id_juego INT,
    IN p_id_biblioteca INT,
    IN p_fecha_adquisicion DATE
)
BEGIN
    INSERT INTO Juegos_Bibliotecas (id_juego, id_biblioteca, fecha_adquisicion)
    VALUES (p_id_juego, p_id_biblioteca, p_fecha_adquisicion);
```

END

RegistrarUsuario: registra nuevos usuarios en la tabla Usuarios, asegurando que el email no esté en uso y configurando valores predeterminados. Este procedimiento utiliza Control de Transacciones (TCL)

```
CREATE PROCEDURE `RegistrarUsuario`(  
    IN p_nombre VARCHAR(100),  
    IN p_email VARCHAR(255),  
    IN p_pais VARCHAR(100)  
)  
BEGIN  
    DECLARE usuario_existente INT DEFAULT 0;  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Error en el registro del usuario durante la transacción.';  
    END;  
    START TRANSACTION;  
    -- Verificar si el email ya está en uso  
    SELECT COUNT(*) INTO usuario_existente  
    FROM Usuarios  
    WHERE email = p_email;  
    IF usuario_existente > 0 THEN  
        ROLLBACK;  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'El email ya está en uso.';  
    ELSE  
        -- Insertar el nuevo usuario  
        INSERT INTO Usuarios (nombre, email, fecha_registro, pais, nivel_cuenta)
```

```
VALUES (p_nombre, p_email, CURDATE(), p_pais, 1);  
  
COMMIT;  
  
END IF;  
  
END;
```

Triggers

juegos_bibliotecas BEFORE INSERT: Este trigger se activará antes de insertar un registro en Juegos_Bibliotecas y verificará que la fecha de adquisición no sea una fecha futura.

```
CREATE TRIGGER `juegos_bibliotecas_BEFORE_INSERT` BEFORE INSERT ON `juegos_bibliotecas` FOR  
EACH ROW BEGIN  
  
IF NEW.fecha_adquisicion > CURDATE() THEN  
  
    SIGNAL SQLSTATE '45000'  
  
    SET MESSAGE_TEXT = 'La fecha de adquisición no puede ser una fecha futura.';  
  
END IF;  
  
END
```

actualizar_nivel_cuenta_BEFORE_INSERT: Este trigger incrementa el nivel de cuenta del usuario cada vez que un juego es agregado a su biblioteca.

```
DELIMITER //  
  
CREATE TRIGGER actualizar_nivel_cuenta_BEFORE_INSERT  
  
AFTER INSERT ON Juegos_Bibliotecas  
  
FOR EACH ROW  
  
BEGIN  
  
    UPDATE Usuarios  
  
    SET nivel_cuenta = nivel_cuenta + 1  
  
    WHERE id_usuario = (SELECT id_usuario FROM Bibliotecas WHERE id_biblioteca =  
NEW.id_biblioteca);  
  
END //
```

DELIMITER ;

Script en SQL de creación de Vistas, Funciones, Stored Procedures y Triggers

En el siguiente link se comparte un repositorio github con el script en SQL de creación de las Vistas, Funciones, Stored Procedures y Triggers:

<https://github.com/juanmamelos/idea-melos.git>

Herramientas y tecnologías usadas

En el desarrollo de este proyecto se utilizaron las siguientes herramientas y tecnologías:

MySQL: Utilizado como sistema de gestión de bases de datos relacional. Proporciona una plataforma robusta y escalable para el manejo y consulta de datos.

MySQL Workbench: Herramienta de diseño y administración de bases de datos que facilitó la creación de modelos E-R, la escritura de consultas SQL, y la administración de la base de datos.

GitHub: Se empleó GitHub como plataforma para alojar el código y la documentación.

Mockaroo: Utilizado para generar datos de prueba realistas. Esta herramienta facilitó la creación de un dataset sintético pero creíble, que simula datos de usuarios, juegos, reseñas y otros elementos clave de la plataforma Steam, lo que permitió probar las funcionalidades de la base de datos en condiciones similares a un entorno de producción.

Futuras líneas

El proyecto de recreación de la base de datos de Steam tiene potencial para ser ampliado y mejorado en futuras iteraciones. Algunas de las posibles líneas de trabajo futuras incluyen:

Seguridad y Control de Accesos: Desarrollar políticas de seguridad más estrictas, incluyendo la implementación de roles y permisos específicos para diferentes tipos de usuarios en la base de datos.

Análisis de Datos Avanzado: Ampliar el alcance del proyecto mediante la incorporación de herramientas de análisis de datos avanzadas para generar insights más profundos a partir de los datos almacenados.